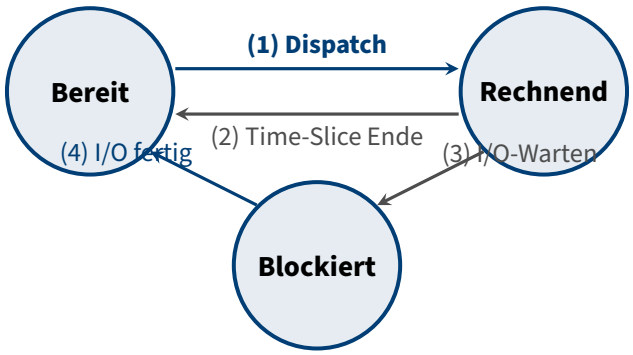
Aufgabe 1: Arbeitszustände von Prozessen

Prozesszustände & Transitionen – Übungsblatt

In Betriebssystemen durchlaufen Prozesse verschiedene Zustände während ihres Lebenszyklus.
Die drei klassischen Zustände: Bereit (Ready): Prozess wartet im Hauptspeicher auf Zuteilung der CPU durch den Scheduler. | **Rechnend (Running):** Prozess führt aktuell Befehle auf der CPU aus. | **Blockiert (Blocked):** Prozess wartet auf ein externes Ereignis (z. B. E/A-Abschluss, Signal).

1.) Klassisches 3-Zustands-Modell und Fehlende Transitionen:

Im folgenden Diagramm sind vier von sechs möglichen Transitionen eingezeichnet:



Geben Sie für die **beiden übrigen Transitionen** jeweils an, ob und unter welchen Umständen diese in einem Betriebssystem sinnvoll bzw. möglich sind:

a) Transition: Bereit → Blockiert (Ready → Blocked)

Nicht möglich. Ein rechenbereiter Prozess befindet sich in der Warteschlange und führt keinen Code aus. Er kann keine E/A-Operation anfordern oder auf eine Ressource warten, ohne vorher gerechnet zu haben.

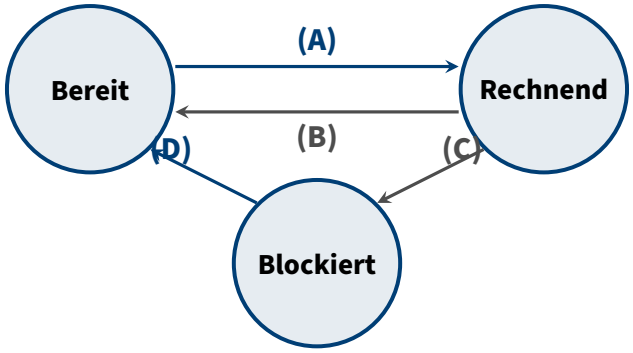
b) Transition: Blockiert → Rechnend (Blocked → Running)

Theoretisch denkbar, in Standard-Betriebssystemen jedoch vermieden. Der Zwischenschritt über *Bereit* stellt sicher, dass der Scheduler über die CPU-Zuteilung gemäß Prioritäten entscheidet. Ein direkter Sprung nach *Rechnend* wäre nur denkbar, wenn die CPU gerade im Leerlauf (Idle) ist.

Simulator ↗ *(Interaktives Prozesszustandsmodell mit Event-Tracing)*

2.) Ereignis-Zuordnung der Transitionen:

Ordnen Sie den Transitionen **(A)**, **(B)**, **(C)** und **(D)** im Zustandsdiagramm die zutreffenden System-Ereignisse zu:

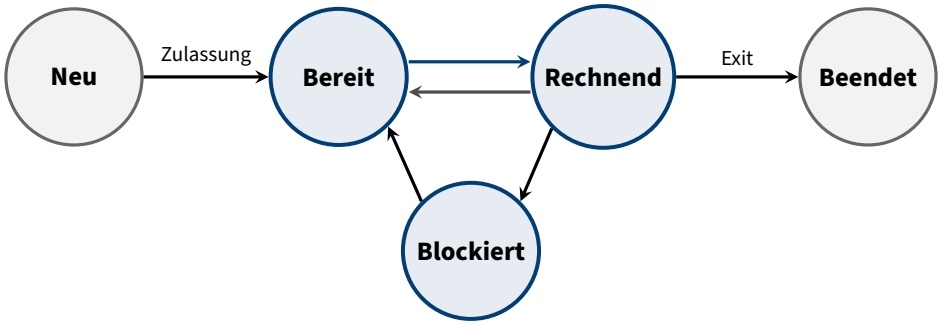


Auslösendes Betriebssystem-Ereignis	Transition
Der Scheduler teilt dem Prozess die CPU zur Ausführung zu	**(A)**
Ein Hardware-Timer löst einen Interrupt aus (Zeitscheibe abgelaufen)	**(B)**
Der Prozess fordert ein blockierendes E/A-Gerät oder Semaphor an	**(C)**
Ein E/A-Controller signalisiert Datenverfügbarkeit via Hardware-Interrupt	**(D)**

Simulator ↗ *(Interaktives Prozesszustandsmodell mit Event-Tracing)*

3.) Erweitertes 5-Zustands-Modell (Neu und Beendet):

In realen Betriebssystemen wird das 3-Zustands-Modell häufig um **Neu (New)** und **Beendet (Terminated)** erweitert:



a) Was geschieht bei der Transition `Neu` → `Bereit` (Zulassung / Admission)?

Der Prozess-Kontrollblock (PCB) wird initialisiert, virtueller Adressraum wird angelegt und der Prozess wird in die Ready-Queue des Schedulers eingereiht.

b) Warum verbleibt ein Prozess nach Beendigung kurzzeitig im Zustand `Beendet` (Zombie-Prozess)?

Damit der Elternprozess den Exit-Status und Rückgabewert (z. B. via `wait()`) abfragen kann. Erst danach wird der PCB endgültig aus der Prozesstabelle gelöscht.

Simulator ↗ *(Interaktives Prozesszustandsmodell mit Event-Tracing)*

4.) Prozess-Lebenszyklus und Ablauf-Trace:

Ein Prozess wird gestartet und durchläuft während seiner Ausführung folgende Aktionen:

1. Erzeugung des Prozesses im System
2. Zuteilung der CPU und Rechnen für 10 ms
3. Anforderung einer Dateilese-Operation von Festplatte
4. Abschluss des Lesevorgangs und Zuteilung der CPU (5 ms)

Geben Sie die lückenlose Kette aller durchlaufenen Prozesszustände (5-Zustands-Modell) an:

`Neu` → `Bereit` → `Rechnend` → `Blockiert` → `Bereit` → `Rechnend` → `Beendet`

Erläutern Sie, warum nach Schritt 3 der Zustand *Bereit* zwingend vor *Rechnend* besucht werden muss:

Wenn die E/A abgeschlossen ist, kann auf der CPU gerade ein anderer Prozess aktiv sein. Der aufgeweckte Prozess muss sich daher in die Bereit-Warteschlange einreihen und auf die Zuteilung durch den Scheduler warten.

Simulator ↗ *(Interaktives Prozesszustandsmodell mit Event-Tracing)*

5.) 7-Zustands-Modell (Auslagerung / Swapping):

Bei akutem Arbeitsspeichermangel kann das Betriebssystem Prozesse auf den Hintergrundspeicher auslagern (Swapping / 7-Zustands-Modell):

a) Warum lagert das Betriebssystem bevorzugt Prozesse aus dem Zustand *Blockiert* statt *Bereit* aus?

Blockierte Prozesse warten auf externe Ereignisse und können ohnehin nicht rechnen. Sie belegen wertvollen RAM, ohne Nutzen zu stiften. Das Auslagern blockierter Prozesse schafft maximalen freien Speicher für Prozesse, die sofort rechnen können.

b) Was geschieht, wenn für einen Prozess im Zustand *Blockiert/Ausgelagert* das ersehnte E/A-Ereignis eintrifft?

Der Prozess wechselt in den Zustand **Bereit/Ausgelagert (Ready/Suspended)**. Er ist nun rechenbereit, befindet sich jedoch weiterhin auf dem Auslagerungsspeicher, bis ihn der Medium-Term-Scheduler wieder in den RAM einlagert.

Simulator ↗ *(Interaktives Prozesszustandsmodell mit Event-Tracing)*